CARS SALES MANAGEMENT SYSTEM

Indian Institute of Information Technology Vadodara

Akshay Kumar 20251651012

Ankur Singh 20251651023

Anurag Sharma 20251651027

Mohit Tailor 20251651058

CS446 Database Management System

Instructor (Dr. Pramit Mazumdar)

Date of submission:

17/04/2026

Table of Contents

1. Abstract

The Car Sales Management System is a full-stack web application developed to digitize and streamline the process of managing car inventories, dealer networks, customer records, sales transactions, and customer feedback. The system was designed as a practical implementation of relational database concepts learned in the Database Management Systems course.

The backend is powered by Node.js and Express.js, connected to a PostgreSQL relational database via the pg driver. Data is organized across six normalized tables — State, Dealer, Car, Customer, Sales, and Feedback — with carefully defined foreign-key relationships to maintain referential integrity. The application exposes a RESTful API layer that supports Create, Read, Update, and Delete (CRUD) operations on every entity in the system.

An analytics dashboard module aggregates key business metrics in real time, including total revenue, top-selling car brands, state-wise revenue distribution, and monthly sales trends. This project demonstrates the integration of database design, backend development, and API engineering to produce a production-grade data management solution.

2. Introduction

The automotive retail industry involves a large volume of interconnected data — from vehicle inventories and dealer networks to customer purchases and post-sale feedback. Managing this data manually through spreadsheets or isolated records leads to redundancy, inconsistency, and difficulty in generating actionable insights. A centralized, database-driven system is essential to ensure data integrity, efficient querying, and real-time reporting.

The Car Sales Management System addresses these challenges by providing a unified platform where administrators can manage all aspects of a car dealership network. The system covers the complete lifecycle of a car sale: from registering vehicles and onboarding dealers, to recording customer purchases and collecting post-sale feedback.

This project was developed as part of the DBMS course to apply core concepts such as Entity-Relationship modelling, normalization, SQL queries with multi-table JOINs, and relational integrity constraints in a real-world scenario. The backend API architecture also reflects software engineering best practices including separation of concerns, modular routing, and environment-based configuration.

2.1 Project Objectives

- Design a normalized relational database schema for a car sales business domain.
- Implement a Node.js/Express backend exposing a complete RESTful API.
- Perform complex JOIN-based SQL queries for sales reporting and dashboard analytics.
- Enable full CRUD operations on all entities: Cars, Dealers, Customers, Sales, Feedback, and States.
- Build an analytics module providing real-time business intelligence such as revenue by state and top brands by units sold.

2.2 Scope

The system is scoped for a car dealership company operating across multiple states with multiple authorized dealers. Each dealer can sell multiple car models to registered customers. After a sale, customers can submit feedback on the dealer. The dashboard provides management-level analytics to support decision-making.

3. Technologies Used

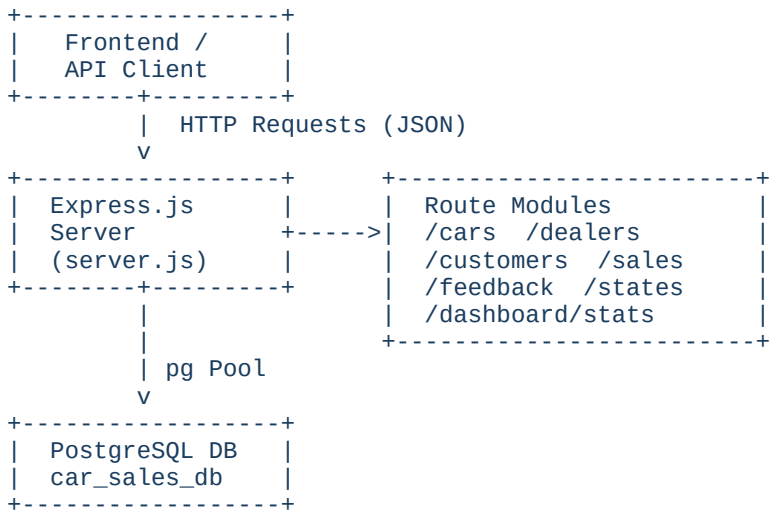
The project adopts a modern web stack with a clear separation between the data layer, business-logic layer, and communication layer. The following table summarizes all technologies used:

Category	Technology	Purpose
Backend Runtime	Node.js v18+	JavaScript runtime for server-side execution
Web Framework	Express.js 4.18	HTTP routing and middleware pipeline
Database	PostgreSQL 15	Relational DBMS for persistent structured data
DB Driver	pg (node-postgres) 8.11	PostgreSQL client library for Node.js
Process Manager	Nodemon 3.0	Auto-restart server on file changes (development)
Config Mgmt	dotenv 16	Loads database credentials from .env file
Cross-Origin	cors 2.8	Enables cross-origin requests from the frontend
API Style	RESTful JSON API	Stateless HTTP API returning JSON responses
Query Language	SQL (PostgreSQL dialect)	Complex JOINs, aggregations, GROUP BY, COALESCE

4. System Architecture

The application follows a three-tier architecture separating the client, server, and database layers. Within the backend, a modular routing pattern is used so that each entity has its own dedicated route file, keeping the codebase organized and maintainable.

4.1 Architecture Overview



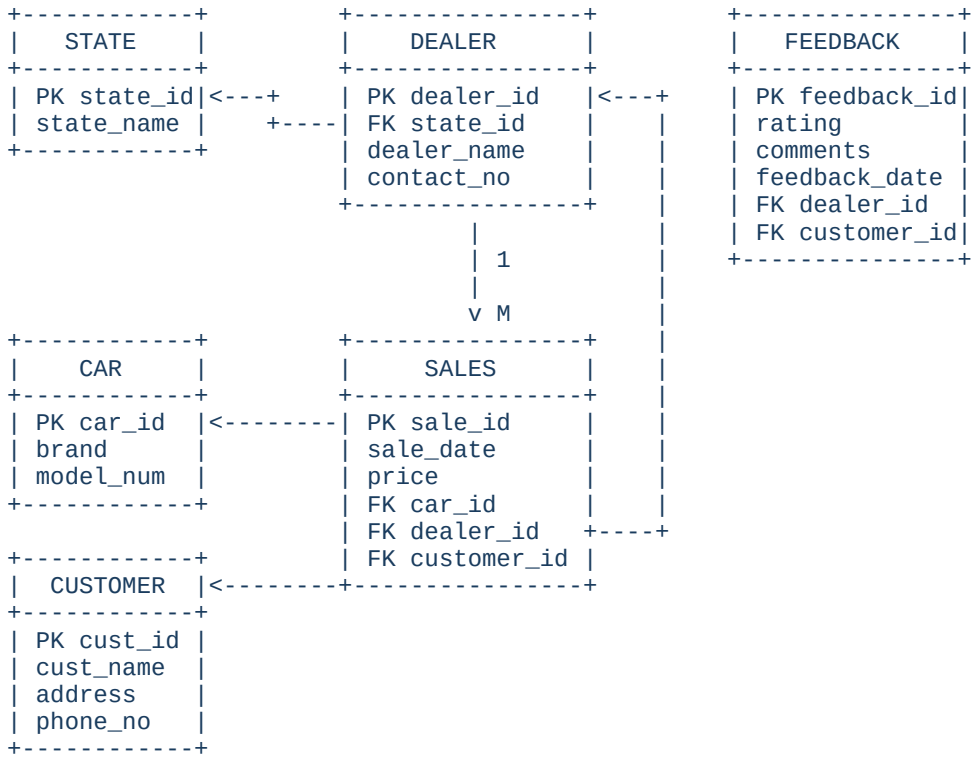
4.2 Backend Modules

- server.js — Application entry point. Initialises Express, registers middleware (cors, json parser), mounts all routers, and starts the HTTP server.
- db.js — Creates and exports a PostgreSQL connection pool using environment variables (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD).
- routes/cars.js — CRUD endpoints for the Car entity.
- routes/dealers.js — CRUD endpoints for the Dealer entity, JOINed with State for display.
- routes/customers.js — CRUD endpoints for Customer; single-customer fetch includes full purchase history via JOIN.
- routes/sales.js — Endpoints for recording and retrieving sales, JOINing Car, Customer, Dealer, and State.
- routes/feedback.js — Endpoints for customer feedback on dealers.
- routes/states.js — CRUD endpoints for geographic State reference data.
- routes/dashboard.js — Single /stats endpoint that runs 8 parallel SQL queries using Promise.all to return all KPIs in one request.

5. ER Diagram

The database schema is designed around six entities. The diagram below shows the entities, their primary keys, foreign keys, and the relationships between them.

5.1 Entity-Relationship Diagram (Textual)



5.2 Entity Descriptions

Entity	Attributes	Description
State	state_id (PK), state_name	Geographic reference table. Each dealer belongs to one state.
Dealer	dealer_id (PK), dealer_name, contact_no, state_id (FK)	An authorized dealership in a specific state. Can make multiple sales.
Car	car_id (PK), brand, model_number	Vehicle catalog. One car model can appear in many sales.
Customer	customer_id (PK), customer_name, address, phone_no	Registered buyer. Can have multiple purchases and submit feedback.
Sales	sale_id (PK), sale_date, price, car_id (FK), dealer_id (FK), customer_id (FK)	Core transactional entity linking a Car, Dealer, and Customer with a price and date.
Feedback	feedback_id (PK), rating, comments, feedback_date, dealer_id (FK), customer_id (FK)	Post-sale rating (1-5) and comments given by a customer about a dealer.

5.3 Key Relationships

- State (1) — (M) Dealer: One state can have many dealers.
- Dealer (1) — (M) Sales: One dealer can facilitate many sales.
- Car (1) — (M) Sales: One car model can be sold in many transactions.
- Customer (1) — (M) Sales: One customer can make multiple purchases.
- Customer (1) — (M) Feedback: One customer can submit feedback for multiple dealers.
- Dealer (1) — (M) Feedback: One dealer can receive multiple feedback entries.

6. Working Flow

The system operates through a stateless RESTful API. Every client action — whether performed by a human via a frontend or by an automated system — sends an HTTP request to the Express server, which processes the request through the appropriate route handler, executes SQL against PostgreSQL, and returns a JSON response.

6.1 End-to-End Workflow

```
CLIENT                  EXPRESS SERVER              PostgreSQL
------                  --------------              ----------
HTTP GET /cars  -----> router -> cars.js  ------> SELECT * FROM car
                                            <------ rows[]
               <----- JSON response

HTTP POST /sales ----> router -> sales.js ------> INSERT INTO sales ...
                                            <------ new row
               <----- 201 Created + JSON

GET /dashboard/stats -> dashboard.js       ------> 8 parallel queries
                  Promise.all([...])        <------ aggregated results
               <----- KPI JSON object
```

6.2 Core User Flows

6.2.1 Adding a New Sale

1. Administrator sends POST /sales with { sale_date, price, car_id, dealer_id, customer_id }.
2. Express validates the body and passes parameters to the PostgreSQL INSERT statement.
3. Foreign-key constraints ensure that the referenced car_id, dealer_id, and customer_id all exist.
4. On success, the new sale row is returned with HTTP 201; on violation, PostgreSQL returns an error that Express forwards as HTTP 500 with the error message.

6.2.2 Viewing Customer Purchase History

A GET /customers/:id request first fetches the customer record, then runs a second JOIN query across sales, car, dealer, and state tables to retrieve the full purchase history sorted by date descending. The combined result is returned as a single JSON object.

6.2.3 Dashboard Analytics

A single GET /dashboard/stats request triggers eight SQL queries in parallel using Promise.all. These queries compute: total sales count, total revenue, total cars in catalog, total customers, total dealers, revenue by state (top 6), top 5 brands by units sold, and monthly sales for the current year. All results are combined into one JSON payload for display.

6.3 API Endpoints Summary

Method	Endpoint	Description

GET	/api/cars	Retrieve all cars ordered by brand
POST	/api/cars	Add a new car model
PUT	/api/cars/:id	Update car brand or model number
DELETE	/api/cars/:id	Remove a car record
GET	/api/dealers	Retrieve all dealers with state name
POST	/api/dealers	Onboard a new dealer
GET	/api/customers/:id	Get customer + full purchase history
POST	/api/customers	Register a new customer
GET	/api/sales	All sales with full JOIN detail
POST	/api/sales	Record a new sale transaction
DELETE	/api/sales/:id	Cancel/delete a sale record
GET	/api/feedback	All feedback with customer & dealer info
POST	/api/feedback	Submit new dealer feedback
GET	/api/dashboard/stats	Full business analytics KPI payload
GET	/api/states	List all states

7. Database Design & Normalization

7.1 Normalization

The schema is normalized to Third Normal Form (3NF):

- 1NF: Every column holds atomic values; no repeating groups.
- 2NF: All non-key attributes are fully dependent on the entire primary key (no partial dependencies, since all PKs are single-column surrogates).
- 3NF: No transitive dependencies exist. For example, state_name is stored only in the State table and referenced via state_id FK in Dealer, rather than duplicating the name.

7.2 Key SQL Patterns Used

- Multi-table JOIN — Sales route joins four tables (sales, customer, car, dealer, state) in a single query to return enriched records.
- LEFT JOIN with COALESCE — Dashboard revenue query uses LEFT JOIN so states with zero sales still appear, and COALESCE(SUM(price), 0) ensures no NULL revenue values.
- Promise.all parallelism — All eight dashboard queries execute concurrently, significantly reducing response latency compared to sequential execution.
- Parameterized queries — All queries use $1, $2 placeholders (not string concatenation) to prevent SQL injection.

8. Conclusion

The Car Sales Management System successfully delivers a functional, normalized, and scalable solution for managing the full data lifecycle of a car dealership business. The project demonstrates the practical application of core database management concepts — including ER modelling, normalization, relational integrity, and multi-table SQL queries — in a real-world software system.

By implementing the backend with Node.js, Express.js, and PostgreSQL, the project also illustrates how a well-designed relational schema can be cleanly integrated with a modern REST API. The modular route structure ensures each domain entity is managed independently, making the codebase easy to extend. The analytics dashboard module showcases how SQL aggregation functions can deliver meaningful business intelligence with minimal additional infrastructure.

Future enhancements to the system could include a React-based frontend dashboard with chart visualizations, user authentication and role-based access control (admin vs. viewer), additional reporting features such as dealer performance rankings, and support for inventory tracking to prevent overselling.